Kashif AI Pipeline Documentation

Overview

This document explains the AI-powered pothole detection and dimension estimation pipeline, from the moment a user submits a report with an image through the mobile app.

Pipeline Flow Diagram

```
┌─────────────────────────────────────────────────────────────────────────────┐
│                                MOBILE APP                                   │
│                                                                             │
│  📱 User takes photo of pothole → Submits report                            │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                             API GATEWAY (nginx)                             │
│                                                                             │
│  🔗 POST /api/reporting/upload                                              │
│                                                                             │
│  Headers: Authorization: Bearer <token>                                     │
│                                                                             │
│  Body: multipart/form-data with image file                                  │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                          REPORTING SERVICE (Port 8000)                      │
│                                                                             │
│  📦 Receives image, validates, saves to /uploads                            │
│                                                                             │
│  Calls AI service for analysis                                              │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          ▼
┌─────────────────────────────────────────────────────────────────────────────┐
│                      POTHOLE DETECTION SERVICE (Port 8006)                  │
│
```

```
│  🤖 /analyze-enhanced endpoint
│
│
│
│  ┌──────────────────────────────────────────────────────────────┐
│  │ Step 1: YOLO Detection (Roboflow API)                        │
│  │ - Model: pothole-clzln/1 (YOLOv8)                            │
│  │ - Detects potholes, returns bounding boxes + confidence      │
│  └──────────────────────────────────────────────────────────────┘
│                              │
│                              ▼
│
│  ┌──────────────────────────────────────────────────────────────┐
│  │ Step 2: Depth Estimation (Replicate API)                     │
│  │ - Model: MiDaS (cjwbw/midas)                                 │
│  │ - Generates depth map from 2D image                          │
│  └──────────────────────────────────────────────────────────────┘
│                              │
│                              ▼
│
│  ┌──────────────────────────────────────────────────────────────┐
│  │ Step 3: Reference Object Detection                           │
│  │ - Looks for credit card, smartphone, etc. in image           │
│  │ - Uses known dimensions for scale calibration                │
│  └──────────────────────────────────────────────────────────────┘
│                              │
│                              ▼
│
│  ┌──────────────────────────────────────────────────────────────┐
│  │ Step 4: Dimension Calculation                                │
│  │ - Combines depth map + reference object + bounding box       │
│  │ - Calculates: width, length, depth, area, volume             │
│  └──────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────────────────────────────────┐
│                        RESPONSE TO APP
│
│  📊 Report stored with AI analysis results
│
│  User sees detection results + dimensions in app
│
└──────────────────────────────────────────────────────────────────┘
```

Step-by-Step Explanation

1. User Submits Report (Mobile App)

Location: `app/` (React Native/Expo)

The user:

1. Opens the Kashif app
2. Takes a photo of a pothole (or selects from gallery)
3. Optionally adds description and location
4. Submits the report

Request:

```
POST /api/reporting/upload
Authorization: Bearer <jwt_token>
Content-Type: multipart/form-data

file: <image.jpg>
```

---

2. Reporting Service Receives Image

Location: `Backend/reporting-service/main.py`

File: `ai_client.py`

The reporting service:

1. Validates the JWT token with auth-service
2. Validates file type (jpeg, png, gif, webp)
3. Validates file size (max 10MB)
4. Saves file to `/app/uploads/` with UUID filename
5. Calls the AI service for analysis

```
# ai_client.py
async def analyze_image(image_path: str, upload_dir: str =
"/app/uploads"):
    async with httpx.AsyncClient(timeout=120.0) as client:
        response = await client.post(
            f"{AI_SERVICE_URL}/analyze-enhanced",
            files=files
        )
```

---

3. Pothole Detection (YOLOv8 via Roboflow)

Location: `Backend/pothole-detection-service/roboflow_detector.py`

Model: `pothole-clzln/1` (YOLOv8 pretrained on pothole dataset)

The detection service:

1. Resizes image to optimal size (960x1280)
2. Sends to Roboflow API
3. Receives bounding boxes with confidence scores

Output:

```
{
  "predictions": [
    {
      "x": 1528,
      "y": 2191.5,
      "width": 1006,
      "height": 897,
      "confidence": 0.401,
      "class": "Potholes"
    }
  ]
}
```

---

4. Depth Estimation (MiDaS via Replicate)

Location: `Backend/pothole-detection-service/depth_estimator.py`

Model: `cjwbw/midas` (MiDaS depth estimation)

The depth estimator:

1. Converts image to base64
2. Sends to Replicate API
3. Receives depth map image
4. Normalizes depth values (0-1 range)

Process:

```
def estimate_midas(self, image_path: str) -> DepthEstimationResult:
    image_uri = self._image_to_base64(image_path)
    result = self._call_replicate("cjwbw/midas", image_uri)
    depth_map = self._download_depth_map(result["output"])
    return DepthEstimationResult(success=True, depth_map=depth_map)
```

Depth Map Example:

- Lighter pixels = closer to camera
- Darker pixels = farther from camera
- Pothole appears darker (deeper) than surrounding surface

---

5. Reference Object Detection

Location: `Backend/pothole-detection-service/dimension_calculator.py`

The system looks for known objects in the image to calibrate scale:

Object	Known Width	Known Height
Credit Card	8.56 cm	5.40 cm
Smartphone	7.0 cm	14.5 cm
Standard Brick	21.5 cm	10.2 cm
A4 Paper	21.0 cm	29.7 cm

Detection Method:

- Color-based detection (credit card edges)
- Shape analysis (rectangular objects)
- Edge detection with Canny

---

6. Dimension Calculation

Location: `Backend/pothole-detection-service/dimension_calculator.py`

Combines all data to calculate real-world dimensions:

```
def calculate_dimensions(
    image: np.ndarray,
    bbox: Tuple[int, int, int, int],
    depth_map: Optional[np.ndarray],
    reference_object: Optional[str]
) -> PotholeDimensions:
```

Calculation Methods:

Method	Confidence	Description
`reference_object`	85%	Uses detected reference object for scale
`depth_based`	65%	Uses depth map gradient analysis
`basic_estimation`	30%	Fallback using camera height assumption

Formulas:

```
width_cm = pixel_width × (reference_width_cm / reference_pixel_width)
depth_cm = (max_depth − pothole_depth) × scale_factor
volume_cm³ = (π/6) × width × length × depth  # Ellipsoid approximation
```

---

7. Response Generation

Location: `Backend/pothole-detection-service/main.py`

The service generates a comprehensive response:

```
{
  "success": true,
  "num_potholes": 1,
  "max_severity": "HIGH",
  "ai_description": "AI detected 1 severe pothole(s). Dimensions: 58.3cm × 52.0cm, depth: 1.0cm, volume: 1585.5cm³.",
  "ai_description_ar": "الذكاء الاصطناعي اكتشف 1 حفرة شديدة...",
  "ai_description_ku": "Zîrekiya destkirdî 1 çalêka giran dît...",
  "detections": [
    {
      "bbox": { "x1": 1025, "y1": 1743, "x2": 2031, "y2": 2640 },
      "confidence": 0.401,
      "severity": "HIGH",
      "estimated_width_cm": 58.3,
      "estimated_height_cm": 52.0,
      "estimated_depth_cm": 1.0,
      "estimated_volume_cm3": 1585.5,
      "dimension_confidence": 0.85,
      "calibration_method": "reference_object"
    }
  ],
  "depth_estimation": {
    "success": true,
    "model_used": "midas",
    "processing_time_ms": 4500
  },
  "annotated_image_base64": "<base64_encoded_image>",
  "depth_map_base64": "<base64_encoded_depth_map>"
}
```

---

8. Report Storage

Location: `Backend/reporting-service/`

The reporting service:

1. Saves annotated image to `/uploads/annotated_<uuid>.jpg`

2. Saves depth map to `/uploads/depth_<uuid>.jpg`
3. Stores AI analysis in report database
4. Returns complete response to mobile app

Severity Classification

Based on pothole dimensions:

Severity	Area (cm²)	Typical Dimensions
LOW	< 500	Small cracks, minor defects
MEDIUM	500 - 2000	Moderate potholes
HIGH	> 2000	Large, dangerous potholes

API Endpoints

Pothole Detection Service

Endpoint	Method	Description
`/health`	GET	Health check
`/detect`	POST	Basic detection (no depth)
`/analyze`	POST	Detection + AI description
`/analyze-enhanced`	POST	Full pipeline with depth estimation

Request Parameters for `/analyze-enhanced`

Parameter	Type	Default	Description
`file`	File	required	Image file (JPEG, PNG, HEIC)
`use_both_depth_models`	bool	false	Use MiDaS only or both models
`detect_reference_object`	bool	true	Enable reference object detection

Environment Variables

```
# Pothole Detection Service
ROBOFLOW_API_KEY=<your_roboflow_key>
REPLICATE_API_TOKEN=<your_replicate_token>

# Reporting Service
AI_SERVICE_URL=http://kashif-pothole-detection:8006
```

Processing Times

Step	Typical Time
Image upload	100-500ms
YOLOv8 detection	500-2000ms
MiDaS depth estimation	3000-60000ms (cold start longer)
Dimension calculation	50-200ms
Total	**4-65 seconds**

> ⚠ First request after cold start may take up to 5 minutes while GPU model initializes on Replicate.

Files Structure

```
Backend/pothole-detection-service/
├── main.py                 # FastAPI endpoints
├── roboflow_detector.py    # YOLOv8 detection via Roboflow
├── depth_estimator.py      # MiDaS depth estimation via Replicate
├── dimension_calculator.py # Real-world dimension calculation
├── enhanced_detector.py    # Combined pipeline orchestrator
├── heic_processor.py       # HEIC image conversion
└── config.py               # Service configuration

Backend/reporting-service/
├── main.py                 # FastAPI endpoints
├── ai_client.py            # Calls pothole-detection service
└── ...
```

Troubleshooting

Depth Estimation Fails

1. Check Replicate API token is set
2. Check Replicate account has credit
3. Increase timeout for cold starts

Low Dimension Confidence

1. Include a reference object (credit card) in photo
2. Take photo from ~1.5m height
3. Ensure good lighting

No Potholes Detected

1. Ensure pothole is clearly visible

2. Minimum confidence threshold is 30%
3. Check image is not blurry

Future Improvements

- [ ] Local depth estimation model (no API dependency)
- [ ] GPS-based scale calibration
- [ ] Multi-pothole tracking
- [ ] Severity prediction based on depth
- [ ] Historical comparison for same location